

Running codes for resolvent/ARNL Madrid project

Alexandra Risha (arisha1@jh.edu)

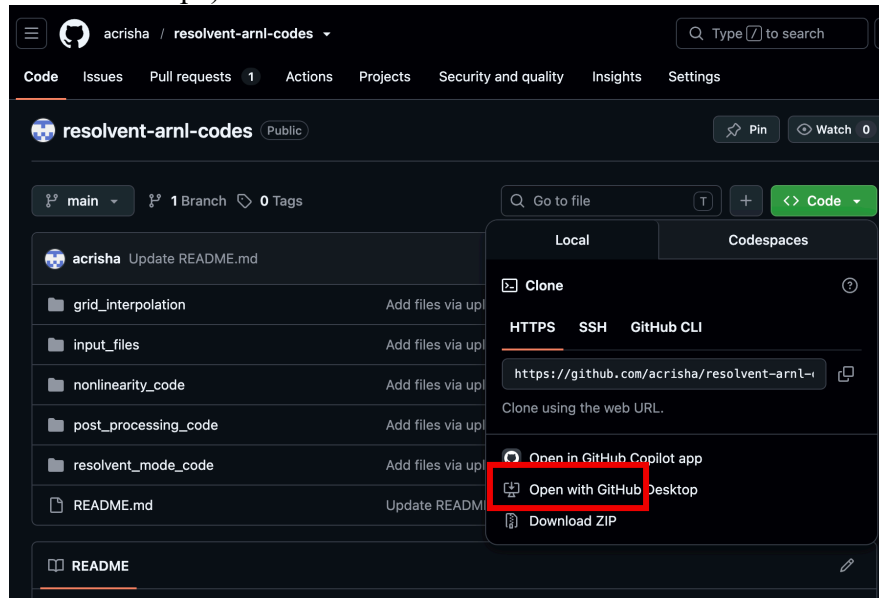
Mechanical Engineering, PhD Student, Johns Hopkins University, Baltimore, MD

Last Modified: July 8, 2026

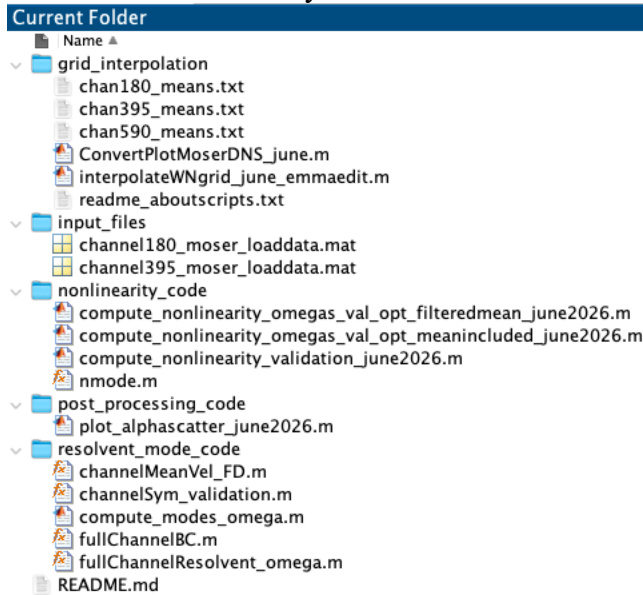
Purpose: This document holds as an outline for first time students to run the resolvent analysis code and nonlinearity codes (currently based in MATLAB).

1. Setting up code from GitHub/OneDrive on local machine

- 1) Go to link to access MATLAB scripts:
<https://github.com/acrisha/resolvent-arnl-codes.git>
- 2) Obtain code from GitHub and put on local machine. You can either use git clone or download the zip file. Downloading is easier (click green “Code” button and then “download zip”).



- 3) Open the “resolvent-arnl-codes” in MATLAB directory of choice. If you expand all the folders this is what you will see:



grid_interpolation: Used to convert online database raw data to the correct wall normal points for the RNLGO code.

input_files: These files are the channel parameters and mean profiles for the different cases we test. These .mat files are also located in the OneDrive folder “Madrid2025-26 ResolventARNL” (you need shared access).

nonlinearity_code: Performs the projection calculation over all triadic interactions, calculates alpha, and sorts alpha. Creates a file you load for post-processing.

post_processing_code: Script used to plot the alpha heat map

resolvent_mode_code: Computes the resolvent modes (which are then used in nonlinearity calculation)

Now you have everything on your local machine you need to run the resolvent modes code and the nonlinearity code. The next 2 sections will discuss a $Re_{\tau} = 395$ case.

2. Resolvent analysis code

- 1) Copy “channel395_moser_loaddata.mat” from **input_files** and paste into **resolvent_mode_code** folder.

Note: The parameter loaddata file changes based on DNS, Reynolds number, channel size etc.

- 2) Go into **resolvent_mode_code** folder and open compute_modes_omega.m

```

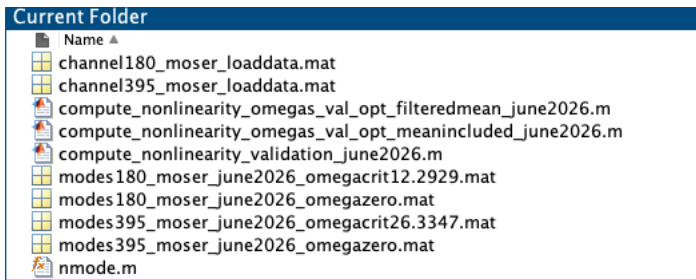
1  clc, clear; close all;
2  % Compute resolvent modes for a span of kx, kz, for a critical omega
3  % x = streamwise, y = wall-normal, z = spanwise
4
5
6  %% user inputs
7  % what Reynolds number channel we want to look at?
8  Retau_target = 395; % 180, 395
9
10 % what omega case do we want to run
11 % 0 for omega = 0
12 % 1 for omega = critical value
13 omegacase = 0;
14
15
16 % use for calculating critical frequency (if omegacase = 1)
17 yplus_crit = 15; % innerscaled wall normal grid point of interest
18 lambdaxplus_crit = 1000; % inner scaled streamwise wavelength associated with wall normal location
19 % ^^ (can use premultiplied energy spectra or literature to determine these)
20
21
22 checkdivergence = false; % do you want to check divergence and output in command window? % normally used for debugging
23
24
25
26 %% load in full channel parameters
27 load(['channel', num2str(Retau_target), '_moser_loaddata.mat']); nx=Nx; nz=Nz; ny = Ny;
28
29 % ensure Umean is saved at cell edges!!
30 Umean = 0.5*(Umean + Umean(end:-1:1))/utau; %symmetrize about centerline
31
32 % save original profile parameters (these will be reloaded in each time)
33 yg_original = yg;
34 y_original = y;
35 Umean_original = Umean;
36
37
38 % use for actual mode calculations
39 Nx = nx/2; % only retain half of streamwise grid points
40 Nz = nz/2; % only retain half of spanwise grid points
41 Ny = ny; % total number of edge points in wall-normal direction, including walls % should be same at RNLGO p.z_w (129)
42 Nkeep = 16; % max amount of modes we can keep, Nkeep is number of modes to keep, preferred DIVISIBLE BY 4
43
44
45 % this is used for debugging!!!! (comment out for running full mode calc)
46 Nx = 4;
47 Nz = 2;
48 Ny = ny;
49 Nkeep = 2;
50

```

- 3) In the user inputs section, you can adjust parameters there. Set parameters to be the ones above.
- 4) Lines ~46-49 are used for debugging or playing with the code. You can pick the number of modes you want to just run the code fast. And click run once you pick values. When running a real case, you want to comment those lines out (like it is shown above).
- 5) Depending on what you are running it will save different file names. Omegacrit will change based on the `yplus_crit` and `lambdaxplus_crit` that you chose. Like below:
 -  `modes180_moser_june2026_omegacrit12.2929.mat`
 -  `modes180_moser_june2026_omegazero.mat`
 -  `modes395_moser_june2026_omegacrit26.3347.mat`
 -  `modes395_moser_june2026_omegazero.mat`
- 6) Now you are ready to run the nonlinearity code....

3. Compute nonlinearity code

- 1) Copy “channel395_moser_loaddata.mat” from **input_files** and paste into **nonlinearity_code** folder.
- 2) Move the “XXXXX.mat” from **resolvent_mode_codes** folder to the **nonlinearity_code** folder.
- 3) Now your folder should look like this:



- 4) Now, you are ready to run the nonlinearity code. You need to decide if you want to filter the streamwise mean modes (i.e., $k_x=0$, $k_x'=0$, and $k_x''=0$) or not. To look at the streamwise triadic pairs in particular, open the “XXX_filteredmean_XXX.m” file.
- 5) When you open it, this is what you see. Follow along the comments in the %%user input section and set Retau and the omegacase that you want. Set validationflag = 0 (this is just for debugging and will increase run time). Leave quickplotcheckflag = 1 so you can quickly see results at the end of the computation.

```

1  clc, clear, close all;
2  tStart = tic;
3  % last updated June 23, 2026
4  % This script computes the nonlinearity (alpha coefficient) using
5  % resolvent modes for different channel flows.
6
7  % Inputs needed:
8  % modesXXX_moser_june2026_omegacrit.mat
9  % modesXXX_moser_june2026_omegazero.mat
10
11 % Outputs:
12 % three .mat files saveXXX.mat used for plotting scatter plot of alpha values
13
14
15 % Run with validationflag=1 to check that we are calculating the nonlinear term correctly
16 % Run with omegacase=0,1, or 2 for different combinations of omega=0 and omega=omega_critical
17 % note: u, v, and w are all stored at cell edges
18 % note: (x,y,z) and (u,v,w) are (streamwise, WN, spanwise)
19
20 %~~~~~
21 %% user inputs
22 % what Reynolds number channel we want to look at?
23 Retau = 180; % 180, 395
24
25
26
27 % do we want to validate symmetry and projection are computed correctly?
28 validationflag = 0; % 0 to skip validation, 1 to run validation
29 % validation will double run time (takes long time)
30
31 % what omega case do we want to run: (where omega = omega' + omega'')
32 % 0: all three omega = 0
33 % 1: for first response mode omega=omega_critical, second response mode omega=0 (forcing mode omega=omega_critical)
34 % 2: for first response mode omega=0, second response mode omega=omega_critical (forcing mode omega=omega_critical)
35 omegacase = 0;
36
37 % what omega critical case are we doing?
38 omegacrit_case = 12.2929; % only used if input the critical omega from the mode .mat file
39
40 quickplotcheckflag = 1; % 0 to skip quick plotting check, 1 to plot figure at end
41
42

```

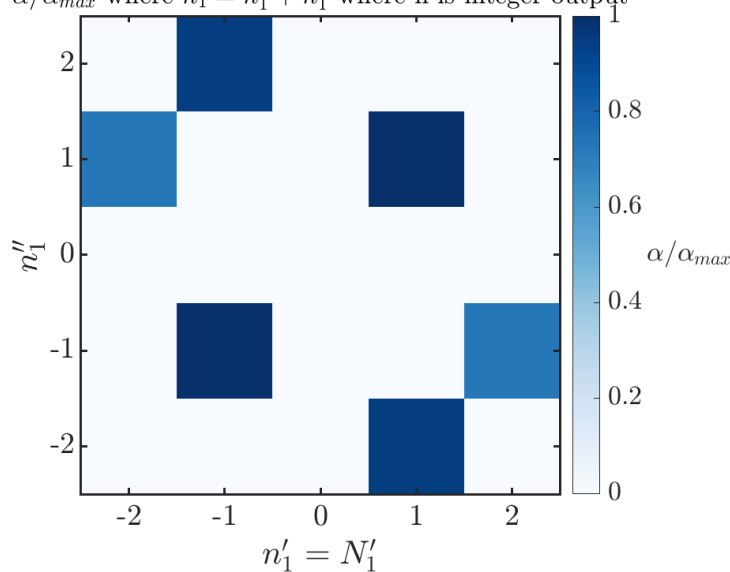
- 6) Scroll in the code to ~lines 59. This is where you will need to adjust things (to play around with the code). Normally, you would want to do all the grid points in the domain, but it just takes too long. So, you can do a manual override and set Nx, Nz, Ny, and the cutoff svd value. You need to at least have the same amount (OR less) Nx, Nz, Ny and svd that you computed using the modes code. *Set (and uncomment the debugging section) like this for now just to run the code fast, but it will not give correct results!!!! You can play around with these variables to see code speed and effect of different parameters.*

```

47 % load in channel parameters
48 load(['channel', num2str(Retau), '_moser_loaddata.mat'], 'Lx', 'Lz', 'Nx', 'Nz', 'Ny', 'y', 'ym');
49 % full channel number of gridpoints
50 nx = Nx; nz = Nz; ny = Ny; clearvars Nx Ny Nz; % clear for consistency (avoid overwriting)
51
52 Nx = nx/2; % streamwise
53 Nz = nz/2; % spanwise
54 Ny = ny; % wall normal
55 nsvd = 2; % max amount of modes we can keep, Nkeep is number of modes to keep, preferred divisibility by 2
56 Nkeep = nsvd;
57
58 %
59 % % % this is used for debugging!!!! (comment out for running full mode calc)
60 % Nx = 4;
61 % Nz = 2;
62 % Ny = ny;
63 % Nkeep = 2;
64 % nsvd = Nkeep; % max amount of modes we can keep, Nkeep is number of modes to keep, preferred divisibility by 2
65

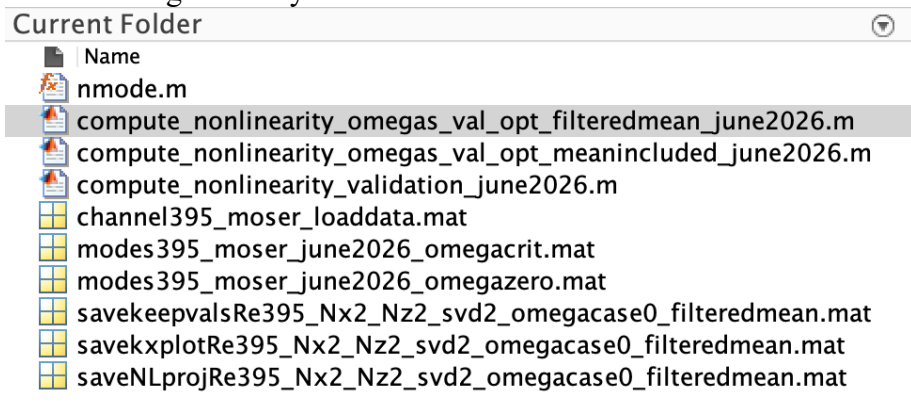
```

- 7) Click run. The command window will help tell you what the code is doing. With the above settings it should take seconds. If quickplotcheckflag = 1 then this will pop up:
 α/α_{max} where $n_1 = n'_1 + n''_1$ where n is integer output



(This is wrong, but what you should be seeing)

- 8) Your working directory will look like this when the simulation is finished:



The saveXX.mat files are used in post processing scripts.

- 9) Open “plot_alphascatter_june2026.m” and fill in parameters from the simulation you ran with the nonlinearity code and then click run. It should create nice heat maps with the integer wave number (NOT THE NONDIMENSIONAL ONE). You can adjust color bars and prettiness later...

